Assignment No 24

Name : Shivam Ramakant Deshmane

Domain : AI & ML

College : T.B Girwalkar Polytechnic Ambajogai

Q1.

```
First 5 Rows:
   Age Sex ChestPainType  RestingBP  Cholesterol  FastingBS RestingECG  MaxHR  \
0   40   M           ATA        140          289          0     Normal    172
1   49   F           NAP        160          180          0     Normal    156
2   37   M           ATA        130          283          0         ST     98
3   48   F           ASY        138          214          0     Normal    108
4   54   M           NAP        150          195          0     Normal    122

  ExerciseAngina  Oldpeak ST_Slope  HeartDisease
0              N      0.0       Up             0
1              N      1.0     Flat             1
2              N      0.0       Up             0
3              Y      1.5     Flat             1
4              N      0.0       Up             0

Missing Values:
Age               0
Sex               0
ChestPainType     0
RestingBP         0
Cholesterol       0
FastingBS         0
RestingECG        0
MaxHR             0
ExerciseAngina    0
Oldpeak           0
ExerciseAngina    0
Oldpeak           0
ST_Slope          0
HeartDisease      0
dtype: int64

Independent Features (X):
Index(['Age', 'Sex', 'ChestPainType', 'RestingBP', 'Cholesterol', 'FastingBS',
       'RestingECG', 'MaxHR', 'ExerciseAngina', 'Oldpeak', 'ST_Slope'],
      dtype='object')

Dependent Feature (y):
HeartDisease

Training Shape: (734, 15)
Testing Shape : (184, 15)
```

Q2.

```
First 5 Rows:
     model  year  price transmission  mileage fuelType  tax   mpg  engineSize
0   Fiesta  2017  12000    Automatic    15944   Petrol  150  57.7         1.0
1    Focus  2018  14000       Manual     9083   Petrol  150  57.7         1.0
2    Focus  2017  13000       Manual    12456   Petrol  150  57.7         1.0
3   Fiesta  2019  17500       Manual    10460   Petrol  145  40.3         1.5
4   Fiesta  2019  16500    Automatic     1482   Petrol  145  48.7         1.0

Missing Values:
model           0
year            0
price           0
transmission    0
mileage         0
fuelType        0
tax             0
mpg             0
engineSize      0
dtype: int64

Independent Features (X):
Index(['model', 'year', 'transmission', 'mileage', 'fuelType', 'tax', 'mpg',
       'engineSize'],
      dtype='object')

Dependent Feature (y):
price

Training Shape: (14249, 34)
Testing Shape : (3563, 34)
```

[3]
0s

```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

[4]
✓ 0s

```
# Load dataset
df = pd.read_csv("heart.csv")

# Remove duplicate rows
df = df.drop_duplicates()

# Independent and Dependent Variables
X = df.drop("HeartDisease", axis=1)
y = df["HeartDisease"]

# One-Hot Encoding
X = pd.get_dummies(X, drop_first=True)

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

# Feature Scaling
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
===== Logistic Regression =====
Accuracy : 0.8532608695652174

Confusion Matrix
[[67 10]
 [17 90]]

Classification Report
              precision    recall  f1-score   support

           0       0.80      0.87      0.83        77
           1       0.90      0.84      0.87       107

    accuracy                           0.85       184
   macro avg       0.85      0.86      0.85       184
weighted avg       0.86      0.85      0.85       184
```

```
===== Decision Tree =====
Accuracy : 0.8260869565217391

Confusion Matrix
[[62 15]
 [17 90]]

Classification Report
              precision    recall  f1-score   support

           0       0.78      0.81      0.79        77
           1       0.86      0.84      0.85       107

    accuracy                           0.83       184
   macro avg       0.82      0.82      0.82       184
weighted avg       0.83      0.83      0.83       184
```

```
===== Support Vector Machine =====
Accuracy : 0.875

Confusion Matrix
[[67 10]
 [13 94]]

Classification Report
              precision    recall  f1-score   support

           0       0.84      0.87      0.85        77
           1       0.90      0.88      0.89       107

    accuracy                           0.88       184
   macro avg       0.87      0.87      0.87       184
weighted avg       0.88      0.88      0.88       184
```

```
===== KNN =====
Accuracy : 0.8532608695652174

Confusion Matrix
[[67 10]
 [17 90]]

Classification Report
              precision    recall  f1-score   support

           0       0.80      0.87      0.83        77
           1       0.90      0.84      0.87       107

    accuracy                           0.85       184
   macro avg       0.85      0.86      0.85       184
weighted avg       0.86      0.85      0.85       184
```

```
===== Naive Bayes =====
Accuracy : 0.8586956521739131

Confusion Matrix
[[68  9]
 [17 90]]

Classification Report
              precision    recall  f1-score   support

           0       0.80      0.88      0.84        77
           1       0.91      0.84      0.87       107

    accuracy                           0.86       184
   macro avg       0.85      0.86      0.86       184
weighted avg       0.86      0.86      0.86       184
```

	Model	Accuracy
2	Support Vector Machine	0.875000
4	Naive Bayes	0.858696
0	Logistic Regression	0.853261
3	KNN	0.853261
1	Decision Tree	0.826087

Q3.

```
1]
0s
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.svm import SVR
from sklearn.neighbors import KNeighborsRegressor

from sklearn.metrics import r2_score
```

[12]
✓ 0s

```
# Load Dataset
df = pd.read_csv("ford.csv")

# Remove duplicate rows
df = df.drop_duplicates()

# Independent and Dependent Variables
X = df.drop("price", axis=1)
y = df["price"]

# One-Hot Encoding
X = pd.get_dummies(X, drop_first=True)

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

# Feature Scaling
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

[13]
✓ 0s

```
lr = LinearRegression()

lr.fit(X_train, y_train)

y_pred_lr = lr.predict(X_test)

print("===== Linear Regression =====")
print("R² Score:", r2_score(y_test, y_pred_lr))
```

```
===== Linear Regression =====
R² Score: 0.8188919779971263
```

0s

```
dt = DecisionTreeRegressor(random_state=42)

dt.fit(X_train, y_train)

y_pred_dt = dt.predict(X_test)

print("===== Decision Tree Regressor =====")
print("R² Score:", r2_score(y_test, y_pred_dt))
```

```
===== Decision Tree Regressor =====
R² Score: 0.8790372530999587
```

[15]
✓ 18s

```
svr = SVR()

svr.fit(X_train, y_train)

y_pred_svr = svr.predict(X_test)

print("===== Support Vector Regressor =====")
print("R² Score:", r2_score(y_test, y_pred_svr))
```

```
===== Support Vector Regressor =====
R² Score: 0.10111342194698003
```

[16]
✓ 0s

```
knn = KNeighborsRegressor(n_neighbors=5)

knn.fit(X_train, y_train)

y_pred_knn = knn.predict(X_test)

print("===== K-Nearest Neighbors Regressor =====")
print("R² Score:", r2_score(y_test, y_pred_knn))
```

```
===== K-Nearest Neighbors Regressor =====
R² Score: 0.9248777341876478
```

```python
comparison = pd.DataFrame({

    "Model": [
        "Linear Regression",
        "Decision Tree Regressor",
        "Support Vector Regressor",
        "K-Nearest Neighbors Regressor"
    ],

    "R² Score": [
        r2_score(y_test, y_pred_lr),
        r2_score(y_test, y_pred_dt),
        r2_score(y_test, y_pred_svr),
        r2_score(y_test, y_pred_knn)
    ]

})

comparison = comparison.sort_values(by="R² Score", ascending=False)

print(comparison)
```

```
                          Model  R² Score
3  K-Nearest Neighbors Regressor  0.924878
1        Decision Tree Regressor  0.879037
0              Linear Regression  0.818892
2       Support Vector Regressor  0.101113
```

Q4.

[18] ✓ 0s

```
import joblib
```

[19] ✓ 0s

```
# Save Best Classification Model
joblib.dump(lr, "best_classification_model.pkl")

# Save Scaler
joblib.dump(scaler, "classification_scaler.pkl")

# Save Column Names
joblib.dump(X.columns.tolist(), "classification_columns.pkl")

print("Best Classification Model Saved Successfully!")
```

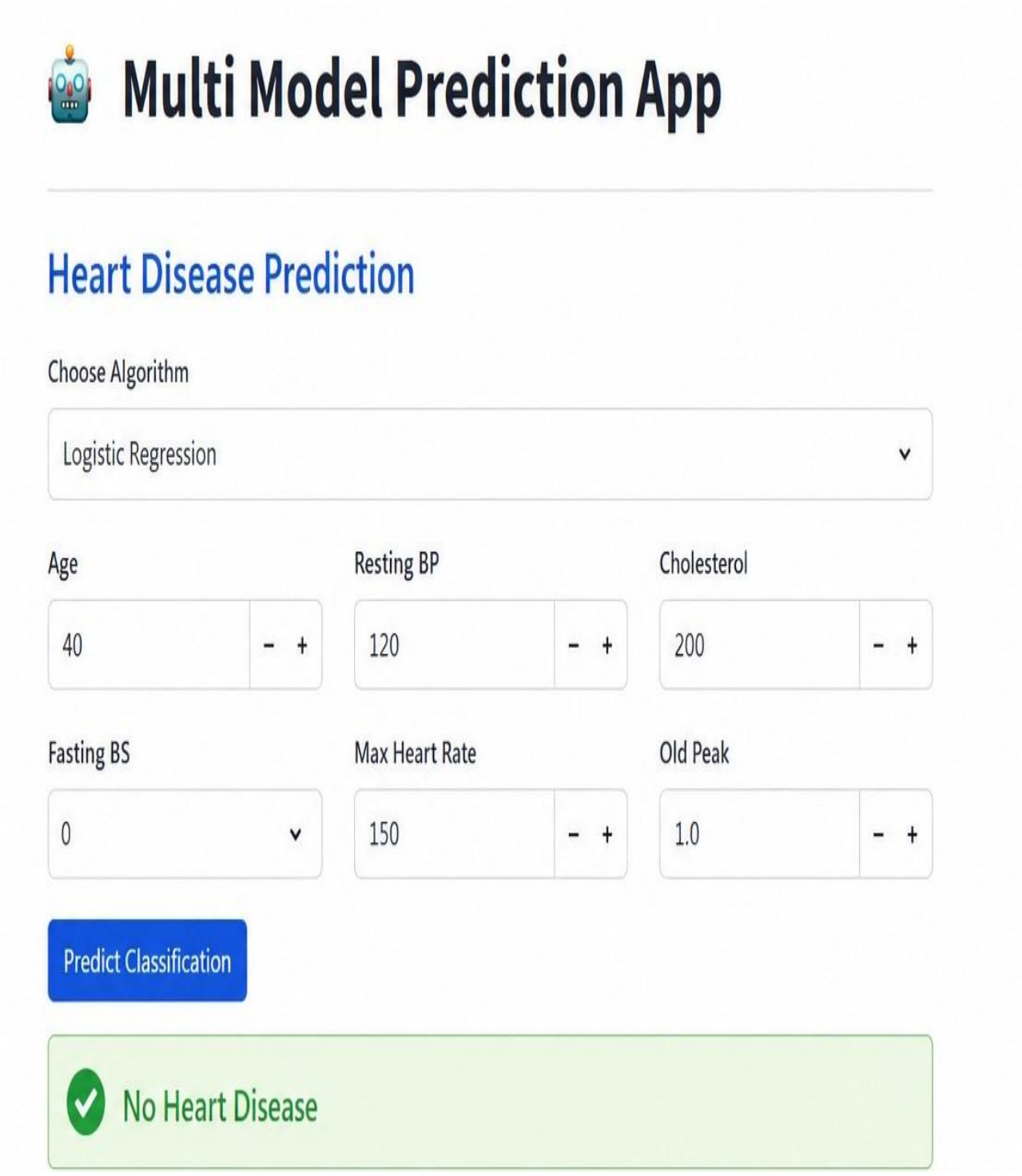
```
Best Classification Model Saved Successfully!
```

[20] ✓ 0s

```
# Save Best Regression Model
joblib.dump(lr, "best_regression_model.pkl")

# Save Scaler
joblib.dump(scaler, "regression_scaler.pkl")

# Save Column Names
joblib.dump(X.columns.tolist(), "regression_columns.pkl")

print("Best Regression Model Saved Successfully!")
```

```
Best Regression Model Saved Successfully!
```

Q5.


Multi Model Prediction App
Heart Disease Prediction
Choose Algorithm
Logistic Regression
Age
40
Resting BP
120
Cholesterol
200
Fasting BS
0
Max Heart Rate
150
Old Peak
1.0
Predict Classification
No Heart Disease